

MetroFlow AI

Full-Stack Tamil Nadu State Traffic Twin & Congestion Predictor

1. Executive Summary

MetroFlow AI is a production-grade, full-stack virtual twin traffic modeling dashboard representing the major state highway corridors and key transit nodes of Tamil Nadu, India. The application simulates vehicular dynamics, handles stochastic road incidents (e.g. accidents, construction blockages), and executes machine learning algorithms on the server to forecast gridlock levels.

The project demonstrates frontend react architectures, high-fidelity responsive dashboard design, vector-based SVG graphics, asynchronous Python web APIs, database ORM integrations, and predictive statistical modelling.

2. Scope & Geographic Coverage

The simulation maps major real-world transit endpoints across the Tamil Nadu state road grid:

- **Chennai (CHN):** Capital transit hub hosting highest vehicular density.
- **Vellore (VEL):** Major northern entry link connecting industrial corridors.
- **Salem (SAL):** Central state highway interchange corridor.
- **Coimbatore (CBE):** Western industrial hub with high freight volumes.
- **Trichy (TRY):** Geographical center nexus linking south and north traffic flows.
- **Madurai (MDU):** Cultural and transit gateway connecting southern districts.
- **Tirunelveli (TNV):** Southern corridor gateway routing to national expressways.

3. System Architecture

MetroFlow AI is designed as a multi-tier decoupled web application:

Layer	Technology	Role & Description
Frontend UI	React, TypeScript, Vite	Renders glassmorphism map dashboard, controls, Recharts trends.
Backend API	FastAPI (Python)	Serves REST routes, manages DB transactions, triggers ML model.
Database	SQLite, SQLAlchemy ORM	Stores history logs and active incidents list. Run as local file.
AI / ML	Scikit-Learn (Random Forest)	Trained on parameters to predict future congestion indexes.

4. Machine Learning Module

Algorithm: Random Forest Regressor (Ensemble Regressor).

Model Input Parameters (Features):

1. **Weather:** Numeric representation (0: Sunny, 1: Rain, 2: Snow).
2. **Time of Day:** Peak multiplier mapping (0: Morning, 1: Midday, 2: Evening, 3: Night).
3. **Event Density:** Surrounding crowds indicator (0: None, 1: Sports, 2: Concert, 3: Holiday).
4. **Active Policies:** Count of dynamic signals, rerouting, and ramp meters deployed.
5. **Incidents:** Number of active road accidents on highways.

Training Pipeline: The `server/train.py` script generates a synthetic historical telemetry dataset (2,500 samples) representing traffic patterns, fits the Random Forest Regressor (Max Depth=8, 100 Estimators), and validates RMSE. The model is exported to `server/model.pkl`. Average model validation accuracy root mean squared error is **5.68%**.

5. Database Schema & Models

SQLite is used via SQLAlchemy ORM. The tables are declared as follows:

Table: traffic_logs

Field Name	Type	Constraint	Description
id	Integer	Primary Key, Auto-increment	Unique identifier for log point.
timestamp	String	Index=True	Datetime string format (e.g. 11:08 AM).
actual	Integer	None	Global congestion index simulated (0-100%).
predicted	Integer	None	ML predicted baseline congestion index.

Table: active_incidents

Field Name	Type	Constraint	Description
id	String	Primary Key	Unique incident GUID (e.g. inc_178254441).
type	String	None	Incident type: accident, construction, breakdown.
roadId	String	None	Highway segment identifier (e.g. nh45_north).
roadName	String	None	Highway name string (e.g. Chennai - Villupuram).
severity	String	None	Severity level: minor, moderate, severe.
description	String	None	Text detailing the road blockage reason.
timestamp	String	None	Spawning timestamp format.

6. API Endpoints List

Route	Method	Request Body	Response Out	Description
/api/history	GET	None	List[TrafficLog]	Retrieves last 15 logs for charting.
/api/logs	POST	TrafficLogCreate	TrafficLog	Writes dynamic telemetry to DB.
/api/incidents	GET	None	List[Incident]	Returns active highway crashes.
/api/incidents	POST	IncidentCreate	Incident	Saves a new incident in DB.
/api/incidents/{id}	DELETE	None	204 No Content	Deletes incident, clearing road.
/api/predict	POST	PredictionInput	JSON	Runs Random Forest ML forecasting.
/api/tomtom	GET	Query params	JSON	Queries TomTom live traffic data.

7. Setup & Run Instructions

Follow these steps to run MetroFlow AI locally:

Step 1: Install Python dependencies

```
pip install -r server/requirements.txt
```

Step 2: Train the Random Forest Regressor

```
python server/train.py
```

Step 3: Run the FastAPI server

```
python server/main.py
```

Starts FastAPI on <http://127.0.0.1:8000>. Docs at <http://127.0.0.1:8000/docs>

Step 4: Install npm modules & start React Frontend

```
npm install npm run dev
```

Launches development dashboard at <http://localhost:5173>

8. Database Utilities CLI Scripts

Manage the database using direct npm command-line triggers:

- **Seed Logs:** `npm run db:seed` (Fills SQLite with 100 historical logs simulating peak curves)
- **Export Data:** `npm run db:export` (Exports all database log records to a CSV file)
- **Clear Database:** `npm run db:clear` (Resets database by deleting logs and active incidents)